OTH REGENSBURG

Praktikum Regelungstechnik (RTP)

Versuch T2: Systemidentifikation

Versuch 1

Gruppe 38
Marius Off
Janis Graf

`Dokument-Version: 02`

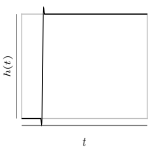
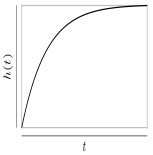
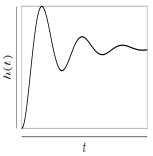
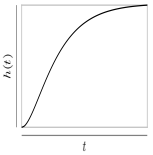
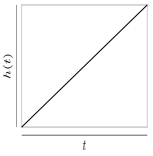
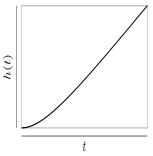
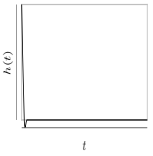
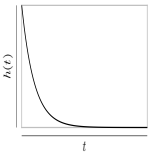
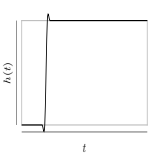
Inhaltsverzeichnis

1 Preperations

In preparation for this experiment, the files provided in ELO were downloaded and saved [**RTP˙Experiments**]. In addition, the step responses for various system behaviours were researched:

1.1 Different basic system behaviors

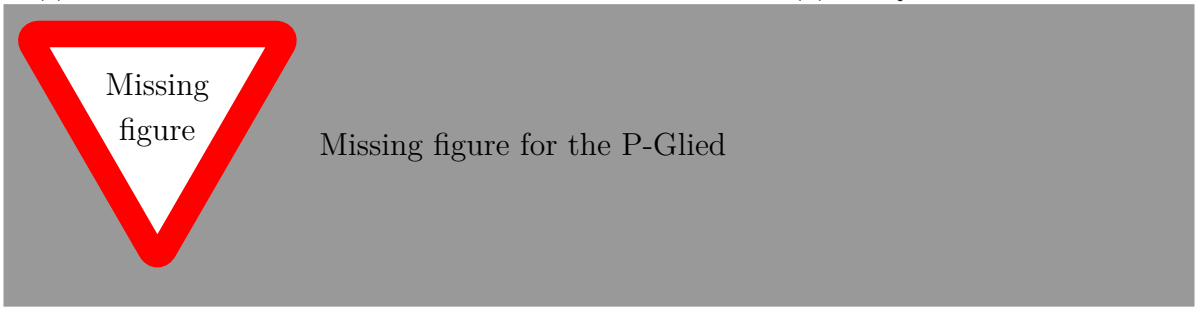
Check names of columns again

Systemverhalten	**Übertragungsfunktion**	**Visualization**
P-Glied (K_p)	$G(s) = K_p$ (1.1)	
PT_1-Glied (K_p, T_1)	$G(s) = \frac{K_p}{T_1 s + 1}$ (1.2)	
PT_2-Glied (K_p, ω_0, D)	$G(s) = \frac{K_p}{\frac{1}{\omega_0^2} s^2 + \frac{2D}{\omega_0} s + 1}$ (1.3)	
PT_2-Glied für $D \geq 1$ (K_p, T_1, T_2)	$G(s) = \frac{K_p}{(T_1 s + 1)(T_2 s + 1)}$ (1.4)	
I-Glied (K_I)	$G(s) = \frac{K_I}{s}$ (1.5)	
IT_1-Glied (K_I, T_1)	$G(s) = \frac{K_I}{s(T_1 s + 1)}$ (1.6)	
D-Glied (K_D)	$G(s) = K_D\, s$ (1.7)	
DT_1-Glied (K_D, T_1)	$G(s) = \frac{K_D\, s}{T_1 s + 1}$ (1.8)	
T_t-Glied (K_p, T_t)	$G(s) = K_p\, e^{-T_t s}$ (1.9)	

1.1.1 P-Glied

Explanation

Proportional-Glied, show a constant behavior in steady state for a constant input. The output is directly proportional to the input, with the proportionality constant being K_p. The equation for a P-Glied is given by $x_a(t) = K_p x_e(t)$, where $x_a(t)$ is the output and $x_e(t)$ is the input. The transfer function of a P-Glied is $G(s) = K_p$.



Missing figure for the P-Glied

Parameters needed for characterization

The only parameter needed to characterize a P-Glied is the proportional gain K_p. This parameter determines the steady-state gain of the system and how much the output will change in response to a change in the input.

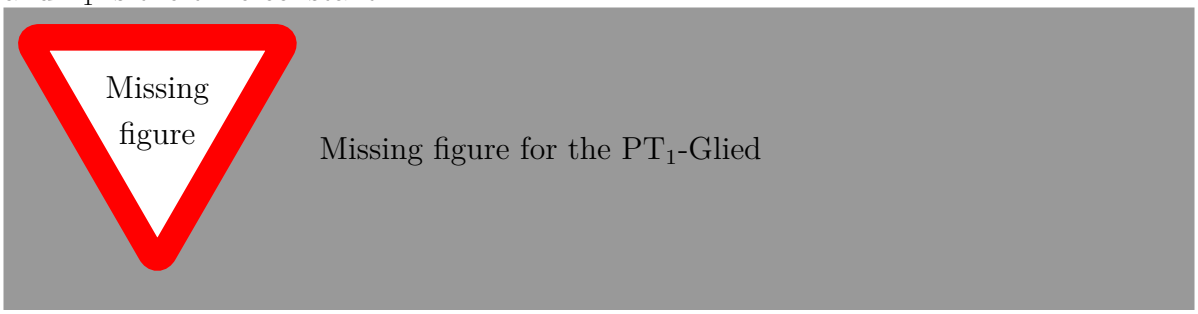
Step and Impulse response

The step response of a P-Glied is a step function that reaches a steady state value of K_p times the input step magnitude. The impulse response is an impulse function scaled by K_p.

1.1.2 PT$_1$-Glied

Explanation

The PT$_1$-Glied is a first-order delay system that consists of a proportional element and a first-order part. The transfer function of a PT$_1$-Glied is given by $T_1\dot{x}_a + x_a(t) = K_p x_e(t)$, or $G(s) = \frac{K_p}{T_1 s + 1}$, where K_p is the proportional gain and the final value of the function and T_1 is the time constant.



Missing figure for the PT$_1$-Glied

Parameters needed for characterization

For the characterization of a PT$_1$-Glied, the parameters needed are the proportional gain K_p and the time constant T_1. The proportional gain K_p determines the steady-state gain of the system, while the time constant T_1 determines how quickly the system responds to changes in the input. The System is also characterized by its initial tangent, which is given by $y(t) = \frac{K_p}{T_1}t$. The point at which the initial tangent intersects the steady state value of the step response can be used to determine the time constant T_1. T_1 is the time it takes for the system to reach approximately 63% of its final value in response to a step input. The threefold multiple of the time constant T_1 at which the system reaches 95% of its final value can also be used to characterize the system's response speed.

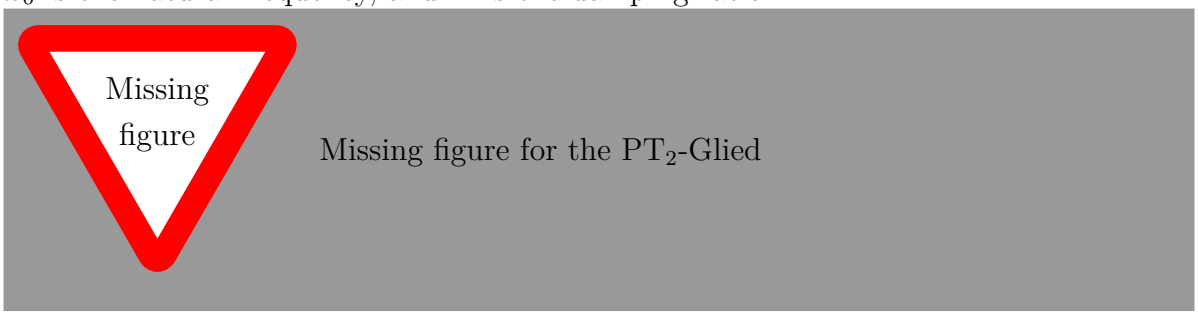
Step and Impulse response

The step response of a PT$_1$-Glied is an exponential function that approaches a steady state value of K_p times the input step magnitude, with a time constant of T_1. The impulse response is an exponential function scaled by K_p/T_1.

1.1.3 PT$_2$-Glied

Explanation

A PT$_2$-Glied is a second-order delay system that consists of a proportional element and a second-order part, which appares delayed and oscillatory behavior becuase of its second order differential nature. The transfer function of a PT$_2$-Glied is given by $\frac{1}{omega_0^2}\ddot{x}_a + \frac{2D}{\omega_0}\dot{x}_a + x_a(t) = K_p x_e(t)$, or $G(s) = \frac{K_p}{\frac{1}{\omega_0^2}s^2 + \frac{2D}{\omega_0}s + 1}$, where K_p is the proportional gain, ω_0 is the natural frequency, and D is the damping ratio.



Missing figure for the PT$_2$-Glied

Parameters needed for characterization

For the characterization of a PT$_2$-Glied, the parameters needed are the proportional gain K_p, the natural frequency ω_0, and the damping ratio D. The proportional gain K_p determines the steady-state gain of the system, while the natural frequency ω_0 and damping ratio D determine the oscillatory behavior and response speed of the system. The damping ratio D indicates whether the system is underdamped ($D < 1$), critically damped ($D = 1$), overdamped ($D > 1$) or undamped ($D = 0$).
For an underdamped system ($D < 1$), the step response will exhibit oscillations before settling to the steady state value. The natural frequency ω_0 determines the frequency of these oscillations, while the damping ratio D determines how quickly the oscillations decay. For a critically damped system ($D = 1$), the step response will reach the steady state value without oscillating, and it will do so as quickly as possible without overshooting. It is often reached by putting two PT$_1$-Glied in series, which results in the transfer function given by $T_1T_2\ddot{x}_a + (T_1 + T_2)\dot{x}_a + x_a(t) = K_p x_e(t)$, or $h(t) = K_P\left(1 - (1 + \omega_0 t)e^{-\omega_0 t}\right)$.
For an overdamped system ($D > 1$), the step response will reach the steady state value without oscillating, but it will take longer to do so than a critically damped system. The Transfer function can be rewritten as $h(t) = K_p\left(1 + \frac{1}{\lambda_1 - \lambda_2}(\lambda_2 e^{-\lambda_1 t} - \lambda_1 e^{-\lambda_2 t})\right)$, where λ_1 and λ_2 are the eigenvalues of the characteristic equation of the system, given by $\lambda_{1,2} = -D\omega_0 \pm \omega_0\sqrt{D^2 - 1}$. The time constants of the system can be determined from these roots, and they indicate how quickly the system responds to changes in the input.
For an undamped system ($D = 0$), the step response will exhibit sustained oscillations at the natural frequency ω_0.
For the case $0 < D < 1$, the transfer function can be rewritten as $h(t) = K_p\left(1 - \frac{e^{-\omega_0 Dt}}{\sqrt{1-D^2}}\sin(\omega_d t + \phi)\right)$.
The Periodic time of the oscillations can be determined from the damped natural frequen-

cy $T_d = \frac{2\pi}{\omega_d} = \frac{2\pi}{\omega_0\sqrt{1-D^2}}$. The Transferfunction can be caracterized by the upper and lower envelope of the oscillations, which are given by $h^{\pm}(t) = K_p(1 \pm \frac{e^- D\omega_0 t}{\sqrt{1-D^2}})$. The over swing time t_{max}, which is the time at which the first maximum of the step response occurs, can be determined from the equation $t_{max} = \frac{\pi}{\omega_d} = \frac{\pi}{\omega_0\sqrt{1-D^2}}$. The overshoot h_{max}, can be determined from the equation $h_{max} = h(t_{max}) = K_p\left(1 + e^{-\pi D/\sqrt{1-D^2}}\right)$. The normalized overshoot value $\ddot{u}$ can be determined from the equation $\ddot{u} = \frac{h_{max}-h(\infty)}{h(\infty)} = e^{\frac{-\pi D}{\sqrt{1-D^2}}}$, which can be used to determine the damping ratio D from the overshoot of the step response. Given by the equation $D = \frac{|\ln(\ddot{u})|}{\sqrt{\pi^2+\ln^2(\ddot{u})}}$. new line

Step and Impulse response

explain two more system behaviors

2 Experiment T2: System identification

2.1 System 0

2.1.1 Experimentaly Identified system parameters

2.1.2 Documentation of the results

Essential Commands

Differential Equation

Transfer Function

2.1.3 Realworld Example

A practical example of such a PT1 system with impulsive excitation is the temperature response of a small metal body to a brief heat flux. If, for example, a metal rod is briefly excited by a heat source (e.g. hot air or a laser) — comparable to a Dirac impulse — the body heats up abruptly and then cools down again via heat conduction and dissipation to the surroundings. The thermal behaviour follows a PT1 model: the temperature rises briefly due to the pulse (corresponding to the step in the output signal) and then falls back to the original level exponentially with a characteristic time constant. The time constant depends on the material properties (heat capacity, thermal conductivity) and the environmental conditions (e.g. convection, radiation). Such system responses frequently occur in sensor technology, materials analysis or in the case of short control pulses in electronic or thermal systems.description of the respective input and output variables.

2.1.4 Comparison of given System with the response of the identified system

Deviations of the System

2.2 System 1

2.2.1 Experimentaly Identified system parameters

2.2.2 Documentation of the results

Essential Commands

Differential Equation

Transfer Function

2.2.3 Realworld Example

description of the respective input and output variables.

2.2.4 Comparison of given System with the response of the identified system

Deviations of the System

2.3 System 2

2.3.1 Experimentaly Identified system parameters

2.3.2 Documentation of the results

Essential Commands

Differential Equation

Transfer Function

2.3.3 Realworld Example

A clear example of an IT1 element with an upstream dead time is a water tank with a constant inflow and proportional outflow. The inflow corresponds to the input signal, and the water level to the output signal. The storage results in integrative behaviour, whilst the proportional outflow causes first-order damping — typical of an IT1 system. In the experimental setup under consideration, a dead-time element was connected upstream of the IT1 element, causing a time delay in the input signal. This means that the system output only reacts to a change after the dead time has elapsed. Such combinations of dead time and IT1 behaviour frequently occur in real-world processes, for example in sluggish piping systems or delayed control variable transmissions. description of the respective input and output variables.

2.3.4 Comparison of given System with the response of the identified system

Deviations of the System

A Appendix A

A.1 List of Figures

Abbildungsverzeichnis

A.2 List of Tables

Tabellenverzeichnis

A.3 List of Variables and Constants used in this report

B Appendix Code and Resources

B.1 Code used for the analysis of the Preperations

Listing B.1: Code used for the analysis of the Preperations

```
{
 "cells": [
  {
   "cell_type": "markdown",
   "id": "b0633634",
   "metadata": {
    "vscode": {
     "languageId": "plaintext"
    }
   },
   "source": [
    "# RTP - System Identification lab exercise\n",
    "\n",
    "\n",
    "Readme.txt exerpt \n",
    "\n",
    "```\n",
    "Data-setup:\n",
    "\n",
    "Real-time measured signals with following setup:\n",
    "\n",
    "first  column: t/s\n",
    "second column: y(t)\n",
    "third  column: x(t)\n",
    "\n",
    "columns separated by: ,\n",
    "decimal separator   : .\n",
    "```"
   ]
  },
  {
```

```
  "cell_type": "code",
  "execution_count": 114,
  "id": "b22b0d1d",
  "metadata": {},
  "outputs": [],
  "source": [
   "import matplotlib.pyplot as plt\n",
   "import math \n",
   "import numpy as np\n",
   "import csv\n",
   "\n"
  ]
 },
 {
  "cell_type": "markdown",
  "id": "ec6391aa",
  "metadata": {},
  "source": [
   "\n",
   "# Task \n",
   "\n",
   "system files:\n",
   "\n",
   "sys0.csv  \n",
   "sys1.csv  \n",
   "sys2.csv\n",
   "\n",
   "\n",
   "\n",
   "2.2 Experiment 2     System identification\n",
   "In this experiment, the parameters of various dynamic systems have to
   "For this purpose, step responses or impulse responses (see xe(t)) wer
   "Document every step thoroughly; show the graphs of the system respons
   "\n",
   "f) Determine the system behavior experimentally using the recorded ste
   "the given systems and their parameters (values and units!). Document
   "essential commands and specify the differential equation and transfer
   "system.\n",
   "g) Provide a real example for each of the determined system behaviors
   "respective input and output variables!\n",
   "h) Compare the given system responses with the responses of your iden
   "graphs and also show the deviation for each system!"
  ]
 },
```

```
{
 "cell_type": "code",
 "execution_count": 115,
 "id": "626299c7",
 "metadata": {},
 "outputs": [
  {
   "data": {
    "image/png": "iVBORw0KGgoAAAANSUhEUgAAAkQAAAHGCAYAAACVcJQUAAAAO
    "text/plain": [
     "<Figure size 640x480 with 1 Axes>"
    ]
   },
   "metadata": {},
   "output_type": "display_data"
  }
 ],
 "source": [
  "# Plot for sys0.csv\n",
  "data0 = np.loadtxt('data/sys0.csv', delimiter=',', skiprows=1)\n",
  "plt.plot(data0[:, 0], data0[:, 2], label='x(t)')\n",
  "plt.plot(data0[:, 0], data0[:, 1], label='y(t)')\n",
  "plt.title('sys0.csv')\n",
  "plt.xlabel('$t/s$')\n",
  "plt.ylabel('$x(t), y(t)$')\n",
  "plt.grid()\n",
  "plt.legend()\n",
  "plt.show()"
 ]
},
{
 "cell_type": "code",
 "execution_count": 116,
 "id": "e2dcdaff",
 "metadata": {},
 "outputs": [
  {
   "data": {
    "image/png": "iVBORw0KGgoAAAANSUhEUgAAAkQAAAHGCAYAAACVcJQUAAAAOnI
    "text/plain": [
     "<Figure size 640x480 with 1 Axes>"
    ]
   },
   "metadata": {},
```

```
    "output_type": "display_data"
   }
  ],
  "source": [
   "# Plot for sys1.csv\n",
   "data1 = np.loadtxt('data/sys1.csv', delimiter=',', skiprows=1)\n",
   "plt.plot(data1[:, 0], data1[:, 2], label='x(t)')\n",
   "plt.plot(data1[:, 0], data1[:, 1], label='y(t)')\n",
   "plt.title('sys1.csv')\n",
   "plt.xlabel('$t/s$')\n",
   "plt.ylabel('$x(t), y(t)$')\n",
   "plt.grid()\n",
   "plt.legend()\n",
   "plt.show()"
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 117,
  "id": "07366d87",
  "metadata": {},
  "outputs": [
   {
    "data": {
     "image/png": "iVBORw0KGgoAAAANSUhEUgAAAj8AAAGHCAYAAACb7AAAAAAAOnI
     "text/plain": [
      "<Figure size 640x480 with 1 Axes>"
     ]
    },
    "metadata": {},
    "output_type": "display_data"
   },
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "Integrated window: 0.00 s to 0.02 s\n",
     "Integral of y(t): 1.000000\n"
    ]
   }
  ],
  "source": [
   "# Integral of sys1 y(t) over the pulse window\n",
   "t_vals = data1[:, 0]\n",
```

```
    "y_vals = data1[:, 1]\n",
    "\n",
    "# Find the non-zero pulse and include the zero samples on both sides,\n",
    "# so the triangle-shaped pulse is integrated over its full 0.02 s wind
    "pulse_mask = y_vals != 0\n",
    "\n",
    "if np.any(pulse_mask):\n",
    "    pulse_indices = np.flatnonzero(pulse_mask)\n",
    "    start_idx = max(pulse_indices[0] - 1, 0)\n",
    "    end_idx = min(pulse_indices[-1] + 1, len(y_vals) - 1)\n",
    "\n",
    "    t_window = t_vals[start_idx:end_idx + 1]\n",
    "    y_window = y_vals[start_idx:end_idx + 1]\n",
    "\n",
    "    integral_y = np.trapezoid(y_window, t_window)\n",
    "\n",
    "    plt.figure()\n",
    "    plt.plot(t_vals, y_vals, label='y(t)')\n",
    "    plt.fill_between(t_window, y_window, alpha=0.25, label='integrated
    "    plt.title('sys1.csv - Integral of y(t)')\n",
    "    plt.xlabel('$t/s$')\n",
    "    plt.ylabel('$y(t)$')\n",
    "    plt.grid()\n",
    "    plt.legend()\n",
    "    plt.show()\n",
    "\n",
    "    print(f'Integrated window: {t_window[0]:.2f} s to {t_window[-1]:.2
    "    print(f'Integral of y(t): {integral_y:.6f}')\n",
    "else:\n",
    "    print('No non-zero samples found in y(t).')"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 118,
   "id": "5e58a18b",
   "metadata": {},
   "outputs": [
    {
     "data": {
      "image/png": "iVBORw0KGgoAAAANSUhEUgAAAjcAAAHGCAYAAACIDqqAAAAACIDqqPAAAAOn
      "text/plain": [
       "<Figure size 640x480 with 1 Axes>"
      ]
```

```
      },
      "metadata": {},
      "output_type": "display_data"
     }
    ],
    "source": [
     "# Plot for sys2.csv\n",
     "data2 = np.loadtxt('data/sys2.csv', delimiter=',', skiprows=1)\n",
     "plt.plot(data2[:, 0], data2[:, 2], label='x(t)')\n",
     "plt.plot(data2[:, 0], data2[:, 1], label='y(t)')\n",
     "plt.title('sys2.csv')\n",
     "plt.xlabel('$t/s$')\n",
     "plt.ylabel('$x(t), y(t)$')\n",
     "plt.grid()\n",
     "plt.legend()\n",
     "plt.show()"
    ]
   },
   {
    "cell_type": "code",
    "execution_count": 119,
    "id": "2448e0cd",
    "metadata": {},
    "outputs": [
     {
      "data": {
       "image/png": "iVBORw0KGgoAAAANSUhEUgAAAkAAAAHFCAYAAAD0bAAAAAOn
       "text/plain": [
        "<Figure size 640x480 with 1 Axes>"
       ]
      },
      "metadata": {},
      "output_type": "display_data"
     },
     {
      "name": "stdout",
      "output_type": "stream",
      "text": [
       "Calculated polyfit function for sys0:\n",
       "            6           5           4          3
2\n",
       "-6.207e-05 x + 0.002433 x - 0.03921 x + 0.3349 x - 1.618 x + 4.27 x
      ]
     }
```

```
   ],
   "source": [
    "# Polynomial fit for x(t) in sys0.csv (exclude initial dead time)\n",
    "t_vals = data0[:, 0]\n",
    "y_vals = data0[:, 1]\n",
    "x_vals = data0[:, 2]\n",
    "\n",
    "sys0_poly_deg = 6\n",
    "\n",
    "# Detect trigger at y(t) step from ~0 to ~2 (crossing midpoint).\n",
    "trigger_candidates = np.where((y_vals[:-1] < 1.0) & (y_vals[1:] >= 1.
    "if len(trigger_candidates) > 0:\n",
    "    trigger_idx = trigger_candidates[0] + 1\n",
    "else:\n",
    "    trigger_idx = 0\n",
    "\n",
    "t_fit = t_vals[trigger_idx:]\n",
    "x_fit_data = x_vals[trigger_idx:]\n",
    "coeffs = np.polyfit(t_fit, x_fit_data, deg=sys0_poly_deg)\n",
    "poly = np.poly1d(coeffs)\n",
    "\n",
    "# Show fit only after trigger so dead time is not included visually.\
    "x_fit = np.full_like(x_vals, np.nan, dtype=float)\n",
    "x_fit[trigger_idx:] = poly(t_fit)\n",
    "\n",
    "plt.figure()\n",
    "plt.plot(t_vals, x_vals, label='x(t) measured')\n",
    "plt.plot(t_vals, x_fit, '--', label=f'polyfit deg={sys0_poly_deg} (po
    "plt.axvline(t_vals[trigger_idx], color='gray', linestyle=':', label='
    "plt.title('sys0.csv - x(t) with dead-time excluded fit')\n",
    "plt.xlabel('t/s')\n",
    "plt.ylabel('x(t)')\n",
    "plt.grid()\n",
    "plt.legend()\n",
    "plt.show()\n",
    "\n",
    "print('Calculated polyfit function for sys0:')\n",
    "print(poly)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 120,
   "id": "882ed43c",
```

```
 "metadata": {},
 "outputs": [
  {
   "data": {
    "image/png": "iVBORw0KGgoAAAANSUhEUgAAAx4AAAE6CAYAAACYmIQ3AAAAOn
    "text/plain": [
     "<Figure size 900x300 with 1 Axes>"
    ]
   },
   "metadata": {},
   "output_type": "display_data"
  },
  {
   "name": "stdout",
   "output_type": "stream",
   "text": [
    "sys0 (deg=6) -> RMSE=0.005634, MAE=0.004517\n"
   ]
  }
 ],
 "source": [
  "# Fit error over time for sys0 (adapts to sys0_poly_deg)\n",
  "deg0 = globals().get('sys0_poly_deg', 6)\n",
  "t0 = data0[:, 0]\n",
  "y0 = data0[:, 1]\n",
  "x0 = data0[:, 2]\n",
  "\n",
  "trigger0_candidates = np.where((y0[:-1] < 1.0) & (y0[1:] >= 1.0))[0]\
  "trigger0_idx = trigger0_candidates[0] + 1 if len(trigger0_candidates)
  "\n",
  "poly0 = np.poly1d(np.polyfit(t0[trigger0_idx:], x0[trigger0_idx:], de
  "x0_fit = np.full_like(x0, np.nan, dtype=float)\n",
  "x0_fit[trigger0_idx:] = poly0(t0[trigger0_idx:])\n",
  "err0 = x0 - x0_fit\n",
  "\n",
  "plt.figure(figsize=(9, 3))\n",
  "plt.plot(t0, err0, label=f'sys0 error (deg={deg0})')\n",
  "plt.axhline(0.0, color='black', linewidth=0.8)\n",
  "plt.axvline(t0[trigger0_idx], color='gray', linestyle=':', label='trig
  "plt.title('sys0 fit error over time (measured - fitted)')\n",
  "plt.xlabel('t/s')\n",
  "plt.ylabel('error')\n",
  "plt.grid()\n",
  "plt.legend()\n",
```

```
    "plt.show()\n",
    "\n",
    "rmse0 = np.sqrt(np.nanmean(err0**2))\n",
    "mae0 = np.nanmean(np.abs(err0))\n",
    "print(f'sys0 (deg={deg0}) -> RMSE={rmse0:.6f}, MAE={mae0:.6f}')"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 121,
   "id": "52685406",
   "metadata": {},
   "outputs": [
    {
     "data": {
      "image/png": "iVBORw0KGgoAAAANSUhEUgAAAj4AAAHFCAYAAADyj/PrAAAAOnRFW
      "text/plain": [
       "<Figure size 640x480 with 1 Axes>"
      ]
     },
     "metadata": {},
     "output_type": "display_data"
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Calculated polyfit function for sys1:\n",
      "            18             17             16             15\n",
      "6.447e+06 x  - 8.908e+07 x  + 5.645e+08 x  - 2.173e+09 x \n",
      "             14             13             12
11\n",
      " + 5.674e+09 x  - 1.062e+10 x  + 1.471e+10 x  - 1.531e+10 x \n",
      "             10             9             8
7\n",
      " + 1.206e+10 x  - 7.198e+09 x + 3.238e+09 x - 1.091e+09 x\n",
      "             6           5             4             3
2\n",
      " + 2.737e+08 x - 5.1e+07 x + 6.942e+06 x - 6.309e+05 x + 2.772e+04 
    ]
   }
  ],
  "source": [
   "# Polynomial fit for x(t) in sys1.csv (no dead time exclusion)\n",
```

```
    "t_vals = data1[:, 0]\n",
    "x_vals = data1[:, 2]\n",
    "\n",
    "sys1_poly_deg = 18\n",
    "\n",
    "coeffs = np.polyfit(t_vals, x_vals, deg=sys1_poly_deg)\n",
    "poly = np.poly1d(coeffs)\n",
    "x_fit = poly(t_vals)\n",
    "\n",
    "\n",
    "plt.figure()\n",
    "plt.plot(t_vals, x_vals, label='x(t) measured')\n",
    "plt.plot(t_vals, x_fit, '--', label=f'polyfit deg={sys1_poly_deg}')\n
    "plt.title('sys1.csv - x(t) with polynomial fit')\n",
    "plt.xlabel('t/s')\n",
    "plt.ylabel('x(t)')\n",
    "plt.grid()\n",
    "plt.legend()\n",
    "plt.show()\n",
    "\n",
    "print('Calculated polyfit function for sys1:')\n",
    "print(poly)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 122,
   "id": "cbcbb0d2",
   "metadata": {},
   "outputs": [
    {
     "data": {
      "image/png": "iVBORw0KGgoAAAANSUhEUgAAAxUAAAE6CAYAAABgeH/AAAAAOnRFW
      "text/plain": [
       "<Figure size 900x300 with 1 Axes>"
      ]
     },
     "metadata": {},
     "output_type": "display_data"
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
```

```
      "sys1 (deg=18) -> RMSE=0.141838, MAE=0.092202\n"
     ]
    }
   ],
   "source": [
    "# Fit error over time for sys1 (adapts to sys1_poly_deg)\n",
    "deg1 = globals().get('sys1_poly_deg', 18)\n",
    "t1 = data1[:, 0]\n",
    "x1 = data1[:, 2]\n",
    "\n",
    "poly1 = np.poly1d(np.polyfit(t1, x1, deg=deg1))\n",
    "x1_fit = poly1(t1)\n",
    "err1 = x1 - x1_fit\n",
    "\n",
    "plt.figure(figsize=(9, 3))\n",
    "plt.plot(t1, err1, label=f'sys1 error (deg={deg1})', color='tab:orange
    "plt.axhline(0.0, color='black', linewidth=0.8)\n",
    "plt.title('sys1 fit error over time (measured - fitted)')\n",
    "plt.xlabel('t/s')\n",
    "plt.ylabel('error')\n",
    "plt.grid()\n",
    "plt.legend()\n",
    "plt.show()\n",
    "\n",
    "rmse1 = np.sqrt(np.nanmean(err1**2))\n",
    "mae1 = np.nanmean(np.abs(err1))\n",
    "print(f'sys1 (deg={deg1}) -> RMSE={rmse1:.6f}, MAE={mae1:.6f}')"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 137,
   "id": "9dbdc803",
   "metadata": {},
   "outputs": [
    {
     "data": {
      "image/png": "iVBORw0KGgoAAAANSUhEUgAAAjMAAAHFCAYAAAAHcXhbAAAAO
      "text/plain": [
       "<Figure size 640x480 with 1 Axes>"
      ]
     },
     "metadata": {},
     "output_type": "display_data"
```

```
   },
   {
    "name": "stdout",
    "output_type": "stream",
    "text": [
     "Calculated polyfit function for sys2:\n",
     "          4          3          2\n",
     "0.4929 x - 5.561 x + 23.29 x - 13.31 x + 0.8802\n"
    ]
   }
  ],
  "source": [
   "# Polynomial fit for x(t) in sys2.csv (exclude initial dead time)\n",
   "t_vals = data2[:, 0]\n",
   "y_vals = data2[:, 1]\n",
   "x_vals = data2[:, 2]\n",
   "\n",
   "sys2_poly_deg = 4\n",
   "\n",
   "trigger_candidates = np.where((y_vals[:-1] < 1.0) & (y_vals[1:] >= 1.
   "if len(trigger_candidates) > 0:\n",
   "    trigger_idx = trigger_candidates[0] + 1\n",
   "else:\n",
   "    trigger_idx = 0\n",
   "\n",
   "t_fit = t_vals[trigger_idx:]\n",
   "x_fit_data = x_vals[trigger_idx:]\n",
   "coeffs = np.polyfit(t_fit, x_fit_data, deg=sys2_poly_deg)\n",
   "poly = np.poly1d(coeffs)\n",
   "\n",
   "x_fit = np.full_like(x_vals, np.nan, dtype=float)\n",
   "x_fit[trigger_idx:] = poly(t_fit)\n",
   "\n",
   "plt.figure()\n",
   "plt.plot(t_vals, x_vals, label='x(t) measured')\n",
   "plt.plot(t_vals, x_fit, '--', label=f'polyfit deg={sys2_poly_deg} (po
   "plt.axvline(t_vals[trigger_idx], color='gray', linestyle=':', label='
   "plt.title('sys2.csv - x(t) with dead-time excluded fit')\n",
   "plt.xlabel('t/s')\n",
   "plt.ylabel('x(t)')\n",
   "plt.grid()\n",
   "plt.legend()\n",
   "plt.show()\n",
   "\n",
```

```
    "print('Calculated polyfit function for sys2:')\n",
    "print(poly)"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 138,
   "id": "c25da406",
   "metadata": {},
   "outputs": [
    {
     "data": {
      "image/png": "iVBORw0KGgoAAAANSUhEUgAAAxUAAAE6CAYAAABgeH/AAAAAOnRFV
      "text/plain": [
       "<Figure size 900x300 with 1 Axes>"
      ]
     },
     "metadata": {},
     "output_type": "display_data"
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "sys2 (deg=4) -> RMSE=0.255885, MAE=0.198389\n"
     ]
    }
   ],
   "source": [
    "# Fit error over time for sys2 (adapts to sys2_poly_deg)\n",
    "deg2 = globals().get('sys2_poly_deg', 4)\n",
    "t2 = data2[:, 0]\n",
    "y2 = data2[:, 1]\n",
    "x2 = data2[:, 2]\n",
    "\n",
    "trigger2_candidates = np.where((y2[:-1] < 1.0) & (y2[1:] >= 1.0))[0]\
    "trigger2_idx = trigger2_candidates[0] + 1 if len(trigger2_candidates)
    "\n",
    "poly2 = np.poly1d(np.polyfit(t2[trigger2_idx:], x2[trigger2_idx:], de
    "x2_fit = np.full_like(x2, np.nan, dtype=float)\n",
    "x2_fit[trigger2_idx:] = poly2(t2[trigger2_idx:])\n",
    "err2 = x2 - x2_fit\n",
    "\n",
    "plt.figure(figsize=(9, 3))\n",
```

```
    "plt.plot(t2, err2, label=f'sys2 error (deg={deg2})', color='tab:green
    "plt.axhline(0.0, color='black', linewidth=0.8)\n",
    "plt.axvline(t2[trigger2_idx], color='gray', linestyle=':', label='trig
    "plt.title('sys2 fit error over time (measured - fitted)')\n",
    "plt.xlabel('t/s')\n",
    "plt.ylabel('error')\n",
    "plt.grid()\n",
    "plt.legend()\n",
    "plt.show()\n",
    "\n",
    "rmse2 = np.sqrt(np.nanmean(err2**2))\n",
    "mae2 = np.nanmean(np.abs(err2))\n",
    "print(f'sys2 (deg={deg2}) -> RMSE={rmse2:.6f}, MAE={mae2:.6f}')"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 125,
   "id": "40a453bc",
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "sys0: best RMSE at degree 19 -> 0.004786\n",
      "sys0: best MAE  at degree 19 -> 0.003802\n",
      "sys1: best RMSE at degree 36 -> 0.114030\n",
      "sys1: best MAE  at degree 36 -> 0.078684\n",
      "sys2: best RMSE at degree 19 -> 0.224799\n",
      "sys2: best MAE  at degree 19 -> 0.176250\n"
     ]
    },
    {
     "data": {
      "image/png": "iVBORw0KGgoAAAANSUhEUgAABEAAAAN6CAYAAAAOhe1AAAAAOnI
      "text/plain": [
       "<Figure size 1100x900 with 3 Axes>"
      ]
     },
     "metadata": {},
     "output_type": "display_data"
    }
   ],
```

```
"source": [
 "# Degree sweep: RMSE/MAE vs polynomial degree (objective degree selec
 "import warnings\n",
 "\n",
 "def sweep_polyfit_metrics(data, degree_range, use_deadtime, name, thr
 "    t = data[:, 0]\n",
 "    y = data[:, 1]\n",
 "    x = data[:, 2]\n",
 "\n",
 "    if use_deadtime:\n",
 "        trigger_candidates = np.where((y[:-1] < threshold) & (y[1:] >=
 "        start_idx = trigger_candidates[0] + 1 if len(trigger_candidat
 "    else:\n",
 "        start_idx = 0\n",
 "\n",
 "    t_fit = t[start_idx:]\n",
 "    x_fit_data = x[start_idx:]\n",
 "\n",
 "    degrees = []\n",
 "    rmses = []\n",
 "    maes = []\n",
 "\n",
 "    for deg in degree_range:\n",
 "        if len(t_fit) <= deg:\n",
 "            continue\n",
 "        try:\n",
 "            with warnings.catch_warnings():\n",
 "                warnings.simplefilter('ignore')\n",
 "                poly = np.poly1d(np.polyfit(t_fit, x_fit_data, deg=de
 "\n",
 "            x_model = np.full_like(x, np.nan, dtype=float)\n",
 "            x_model[start_idx:] = poly(t_fit)\n",
 "            err = x - x_model\n",
 "\n",
 "            degrees.append(deg)\n",
 "            rmses.append(float(np.sqrt(np.nanmean(err**2))))\n",
 "            maes.append(float(np.nanmean(np.abs(err))))\n",
 "        except Exception:\n",
 "            continue\n",
 "\n",
 "    if len(degrees) == 0:\n",
 "        raise ValueError(f'No valid degrees for {name}.')\n",
 "\n",
 "    best_rmse_idx = int(np.argmin(rmses))\n",
```

```
    "    best_mae_idx = int(np.argmin(maes))\n",
    "\n",
    "    print(f'{name}: best RMSE at degree {degrees[best_rmse_idx]} -> {
    "    print(f'{name}: best MAE  at degree {degrees[best_mae_idx]} -> {m
    "\n",
    "    return np.array(degrees), np.array(rmses), np.array(maes)\n",
    "\n",
    "# You can change these ranges whenever you want.\n",
    "deg_range0 = range(1, 20)\n",
    "deg_range1 = range(1, 40)\n",
    "deg_range2 = range(1, 20)\n",
    "\n",
    "d0, r0, m0 = sweep_polyfit_metrics(data0, deg_range0, use_deadtime=Tr
name='sys0')\n",
    "d1, r1, m1 = sweep_polyfit_metrics(data1, deg_range1, use_deadtime=Fa
    "d2, r2, m2 = sweep_polyfit_metrics(data2, deg_range2, use_deadtime=Tr
name='sys2')\n",
    "\n",
    "plt.figure(figsize=(11, 9))\n",
    "\n",
    "plt.subplot(3, 1, 1)\n",
    "plt.plot(d0, r0, '-o', label='RMSE')\n",
    "plt.plot(d0, m0, '-s', label='MAE')\n",
    "plt.title('sys0: metric vs degree')\n",
    "plt.ylabel('metric')\n",
    "plt.grid()\n",
    "plt.legend()\n",
    "\n",
    "plt.subplot(3, 1, 2)\n",
    "plt.plot(d1, r1, '-o', label='RMSE', color='tab:orange')\n",
    "plt.plot(d1, m1, '-s', label='MAE', color='tab:red')\n",
    "plt.title('sys1: metric vs degree')\n",
    "plt.ylabel('metric')\n",
    "plt.grid()\n",
    "plt.legend()\n",
    "\n",
    "plt.subplot(3, 1, 3)\n",
    "plt.plot(d2, r2, '-o', label='RMSE', color='tab:green')\n",
    "plt.plot(d2, m2, '-s', label='MAE', color='tab:olive')\n",
    "plt.title('sys2: metric vs degree')\n",
    "plt.xlabel('degree')\n",
    "plt.ylabel('metric')\n",
    "plt.grid()\n",
    "plt.legend()\n",
```

```
    "\n",
    "plt.tight_layout()\n",
    "plt.show()"
   ]
  },
  {
   "cell_type": "markdown",
   "id": "ad67d9cb",
   "metadata": {},
   "source": [
    "#### Section to calculate all parameters of the systems and to plot th
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 126,
   "id": "9215b455",
   "metadata": {},
   "outputs": [
    {
     "data": {
      "image/png": "iVBORw0KGgoAAAANSUhEUgAABW0AAAJOCAYAAADMCCWlAAAAAC
      "text/plain": [
       "<Figure size 1400x600 with 2 Axes>"
      ]
     },
     "metadata": {},
     "output_type": "display_data"
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Estimated sys0 PT1 parameters:\n",
      "K_P = 0.998876\n",
      "T_t = 0.050000 s\n",
      "T_1 = 1.000000 s\n",
      "3T_1 = 2.890000 s\n",
      "    bertragungsfunktion     (PT1 with dead time):\n",
      "G(s) = (0.998876) / (1.000000 s + 1) * exp(-0.050000 s)\n"
     ]
    }
   ],
   "source": [
```

```
"# sys0 PT1 identification with dead time and annotated plot\n",
"t = data0[:, 0]\n",
"u = data0[:, 1]  # input y(t)\n",
"x = data0[:, 2]  # output x(t)\n",
"\n",
"# Trigger time from input step (0 -> 2).\n",
"trigger_candidates = np.where((u[:-1] < 1.0) & (u[1:] >= 1.0))[0]\n",
"trigger_idx = trigger_candidates[0] + 1 if len(trigger_candidates) >
"t_step = t[trigger_idx]\n",
"\n",
"# Baseline and steady-state values.\n",
"x0 = np.mean(x[:max(1, trigger_idx)])\n",
"x_inf = np.mean(x[-10:])\n",
"u0 = np.mean(u[:max(1, trigger_idx)])\n",
"u_inf = np.mean(u[trigger_idx:])\n",
"du = u_inf - u0\n",
"dx = x_inf - x0\n",
"\n",
"# Static gain Kp = Delta output / Delta input.\n",
"K_p = dx / du if abs(du) > 1e-12 else np.nan\n",
"\n",
"# Work on normalized response after trigger.\n",
"t_rel = t - t_step\n",
"x_norm = (x - x0) / dx if abs(dx) > 1e-12 else np.zeros_like(x)\n",
"\n",
"# Dead time T_t: first time output exceeds 5% of total rise after trig
"post = np.arange(trigger_idx, len(t))\n",
"idx_5 = post[np.where(x_norm[post] >= 0.05)[0][0]] if np.any(x_norm[p
"t_t = t[idx_5] - t_step\n",
"\n",
"# PT1 time markers from normalized rise.\n",
"idx_63 = post[np.where(x_norm[post] >= (1.0 - np.exp(-1.0)))[0][0]] i
"idx_95 = post[np.where(x_norm[post] >= (1.0 - np.exp(-3.0)))[0][0]] i
"\n",
"T1 = t[idx_63] - t_step\n",
"T3 = t[idx_95] - t_step\n",
"\n",
"# 63% and 95% levels in output coordinates.\n",
"x_63 = x0 + (1.0 - np.exp(-1.0)) * dx\n",
"x_95 = x0 + (1.0 - np.exp(-3.0)) * dx\n",
"\n",
"# Tangent from the local origin (t_step, x0), slope from early post-s
"fit_end = min(trigger_idx + 8, len(t))\n",
"if fit_end > trigger_idx + 1:\n",
```

```
"    slope, _ = np.polyfit(t[trigger_idx:fit_end], x[trigger_idx:fit_en
"else:\n",
"    slope = 0.0\n",
"\n",
"# Draw tangent only in the identification region to keep the plot sca
"tangent = np.full_like(t, np.nan, dtype=float)\n",
"tangent_end_time = min(t_step + T3, t[-1])\n",
"tangent_mask = (t >= t_step) & (t <= tangent_end_time)\n",
"tangent[tangent_mask] = x0 + slope * (t[tangent_mask] - t_step)\n",
"\n",
"# PT1 + dead-time model response for comparison.\n",
"x_model = np.where(t_rel >= t_t, x0 + dx * (1.0 - np.exp(-(t_rel - t_
"\n",
"# Two-panel plot: left input, right system response with markers.\n",
"fig, (ax_in, ax_out) = plt.subplots(1, 2, figsize=(14, 6), gridspec_kw
"\n",
"# Left: input function.\n",
"ax_in.plot(t, u, color='tab:orange', label='y(t) input', linewidth=2)
"ax_in.axvline(t_step, color='black', linestyle='--', label='step origi
"ax_in.axhline(u_inf, color='tab:orange', linestyle=':', label=r'$\\ha
"ax_in.set_title('sys0 input signal')\n",
"ax_in.set_xlabel('$t/s$')\n",
"ax_in.set_ylabel('$y(t)$')\n",
"ax_in.grid()\n",
"ax_in.legend(loc='best')\n",
"\n",
"# Right: output and PT1 identification markers.\n",
"ax_out.plot(t, x, label=r'$\\hat{x}(t)$ measured', linewidth=2)\n",
"ax_out.plot(t, tangent, '--', label='tangent from origin (t_step, x0)
"ax_out.plot(t, x_model, ':', label='PT1 + dead-time model')\n",
"\n",
"ax_out.axvline(t_step, color='black', linestyle='--', label='step orig
"ax_out.axvline(t_step + t_t, color='purple', linestyle='--', label='T
"ax_out.axvline(t_step + T1, color='tab:green', linestyle='--', label=
"ax_out.axvline(t_step + T3, color='tab:orange', linestyle='--', label
"\n",
"# Requested rename: show steady-state output line as K_P in the legend
"ax_out.axhline(x_inf, color='tab:red', linestyle='-.', label='K_P')\n
"ax_out.axhline(x_63, color='tab:green', linestyle=':', label='63% leve
"ax_out.axhline(x_95, color='tab:orange', linestyle=':', label='95% lev
"ax_out.scatter([t_step + T1, t_step + T3], [x[idx_63], x[idx_95]], co
"\n",
"# Keep readable axis range based on output signal.\n",
"y_min = np.min(x)\n",
```

```
    "y_max = np.max(x)\n",
    "y_pad = 0.1 * (y_max - y_min) if y_max > y_min else 1.0\n",
    "ax_out.set_xlim(t[0], t[-1])\n",
    "ax_out.set_ylim(y_min - y_pad, y_max + y_pad)\n",
    "\n",
    "ax_out.set_title('sys0: PT1 identification with marked parameters')\n
    "ax_out.set_xlabel('$t/s$')\n",
    "ax_out.set_ylabel(r'$\\hat{x}(t)$')\n",
    "ax_out.grid()\n",
    "ax_out.legend(loc='upper left', bbox_to_anchor=(1.02, 1.0), borderaxes
    "\n",
    "plt.tight_layout()\n",
    "plt.show()\n",
    "\n",
    "print('Estimated sys0 PT1 parameters:')\n",
    "print(f'K_P = {K_p:.6f}')\n",
    "print(f'T_t = {t_t:.6f} s')\n",
    "print(f'T_1 = {T1:.6f} s')\n",
    "print(f'3T_1 = {T3:.6f} s')\n",
    "print('   bertragungsfunktion    (PT1 with dead time):')\n",
    "print(f'G(s) = ({K_p:.6f}) / ({T1:.6f} s + 1) * exp(-{t_t:.6f} s)')"
   ]
  },
  {
   "cell_type": "code",
   "execution_count": 127,
   "id": "928461d4",
   "metadata": {},
   "outputs": [
    {
     "data": {
      "image/png": "iVBORw0KGgoAAAANSUhEUgAABW4AAAJOCAYAAAAnP56mAAAA
      "text/plain": [
       "<Figure size 1400x600 with 2 Axes>"
      ]
     },
     "metadata": {},
     "output_type": "display_data"
    },
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "Estimated sys1 PT2 parameters from original impulse response:\n",
```

```
    "A_u = 1.000000\n",
    "K_p = 1.945454\n",
    "D = 0.254430\n",
    "omega_0 = 19.687860 1/s\n",
    "T_d = 0.330000 s\n",
    "t_max = 0.070000 s\n",
    "x_max = 28.369144\n",
    "x(inf) = 0.027507\n",
    "Transfer function (PT2 form):\n",
    "G(s) = (1.945454) / ((1/19.687860^2) s^2 + (2*0.254430/19.687860) s
   ]
  }
 ],
 "source": [
  "# sys1 PT2 identification from original impulse response x(t)\n",
  "t = data1[:, 0]\n",
  "u = data1[:, 1]  # input y(t), impulse-like\n",
  "x = data1[:, 2]  # measured output x(t)\n",
  "\n",
  "# Trigger detection at first significant input rise.\n",
  "u_min = float(np.min(u))\n",
  "u_max = float(np.max(u))\n",
  "u_mid = 0.5 * (u_min + u_max)\n",
  "trigger_candidates = np.where((u[:-1] < u_mid) & (u[1:] >= u_mid))[0]\
  "trigger_idx = trigger_candidates[0] + 1 if len(trigger_candidates) >
  "t_step = t[trigger_idx]\n",
  "\n",
  "# Input pulse area A_u = integral y(t) dt.\n",
  "A_u = float(np.trapezoid(u, t))\n",
  "if abs(A_u) < 1e-12:\n",
  "    raise ValueError('Input pulse area is too small for PT2 identifica
  "\n",
  "# Steady value for impulse response should be ~0 after decay.\n",
  "tail_start = int(0.9 * len(t))\n",
  "x_inf = float(np.mean(x[tail_start:]))\n",
  "x_rel = x - x_inf\n",
  "\n",
  "# Detect extrema after trigger using slope sign changes.\n",
  "dxdt = np.gradient(x_rel, t)\n",
  "max_idx = []\n",
  "min_idx = []\n",
  "for i in range(max(trigger_idx + 1, 1), len(t) - 1):\n",
  "    if dxdt[i - 1] > 0 and dxdt[i] <= 0:\n",
  "        max_idx.append(i)\n",
```

```
"    if dxdt[i - 1] < 0 and dxdt[i] >= 0:\n",
"        min_idx.append(i)\n",
"max_idx = np.array(max_idx, dtype=int)\n",
"min_idx = np.array(min_idx, dtype=int)\n",
"\n",
"if len(max_idx) == 0:\n",
"    raise ValueError('No positive peak detected in impulse response.')
"\n",
"# First positive peak -> t_max and x_max.\n",
"first_peak_idx = int(max_idx[0])\n",
"t_max_rel = max(t[first_peak_idx] - t_step, 0.0)\n",
"x_max = float(x[first_peak_idx])\n",
"A1 = float(abs(x_rel[first_peak_idx]))\n",
"\n",
"# Period T_d from first two positive peaks; fallback from positive-to-
"if len(max_idx) >= 2:\n",
"    T_d = float(t[max_idx[1]] - t[max_idx[0]])\n",
"elif len(min_idx) >= 1 and t[min_idx[0]] > t[first_peak_idx]:\n",
"    T_d = 2.0 * float(t[min_idx[0]] - t[first_peak_idx])\n",
"else:\n",
"    T_d = np.nan\n",
"\n",
"# Damping D from logarithmic decrement of peak amplitudes.\n",
"if len(max_idx) >= 2:\n",
"    A2 = float(abs(x_rel[max_idx[1]]))\n",
"    if A1 > 1e-12 and A2 > 1e-12 and A1 > A2:\n",
"        delta = np.log(A1 / A2)\n",
"        D = delta / np.sqrt((2.0 * np.pi)**2 + delta**2)\n",
"    else:\n",
"        delta = np.nan\n",
"        D = np.nan\n",
"else:\n",
"    delta = np.nan\n",
"    D = np.nan\n",
"\n",
"# Frequencies.\n",
"omega_d = 2.0 * np.pi / T_d if np.isfinite(T_d) and T_d > 1e-12 else
"if np.isfinite(D) and 0.0 <= D < 1.0 and np.isfinite(omega_d):\n",
"    omega_0 = omega_d / np.sqrt(1.0 - D**2)\n",
"else:\n",
"    omega_0 = np.nan\n",
"\n",
"# Static gain K_p from impulse-to-step relation: K_p = (1/A_u) * inte
"K_p = float(np.trapezoid(x_rel, t) / A_u)\n",
```

```
"\n",
"# Impulse-response model of underdamped PT2 for comparison.\n",
"tau = np.maximum(t - t_step, 0.0)\n",
"if np.isfinite(D) and 0.0 < D < 1.0 and np.isfinite(omega_0):\n",
"    amp = A_u * K_p * omega_0 / np.sqrt(1.0 - D**2)\n",
"    x_model = x_inf + amp * np.exp(-D * omega_0 * tau) * np.sin(omega
"    x_env_plus = x_inf + amp * np.exp(-D * omega_0 * tau)\n",
"    x_env_minus = x_inf - amp * np.exp(-D * omega_0 * tau)\n",
"else:\n",
"    x_model = np.full_like(t, np.nan, dtype=float)\n",
"    x_env_plus = np.full_like(t, np.nan, dtype=float)\n",
"    x_env_minus = np.full_like(t, np.nan, dtype=float)\n",
"\n",
"# Plot: original impulse input and original measured output with PT2
"fig, (ax_in, ax_out) = plt.subplots(1, 2, figsize=(14, 6), gridspec_kw
"\n",
"ax_in.plot(t, u, color='tab:orange', label='y(t) impulse input', linew
"ax_in.axvline(t_step, color='black', linestyle='--', label='trigger')\
"ax_in.set_title('sys1 input signal')\n",
"ax_in.set_xlabel('$t/s$')\n",
"ax_in.set_ylabel('$y(t)$')\n",
"ax_in.grid()\n",
"ax_in.legend(loc='best')\n",
"\n",
"ax_out.plot(t, x, label='x(t) measured', linewidth=2)\n",
"ax_out.plot(t, x_model, ':', label='PT2 impulse model')\n",
"ax_out.plot(t, x_env_plus, '--', color='tab:green', label='envelope +
"ax_out.plot(t, x_env_minus, '--', color='tab:green', label='envelope
"\n",
"ax_out.axvline(t_step, color='black', linestyle='--', label='trigger')
"ax_out.axvline(t[first_peak_idx], color='tab:purple', linestyle='--',
"ax_out.axhline(x_inf, color='tab:red', linestyle='-.', label='x(inf)')
"ax_out.axhline(x_max, color='tab:blue', linestyle=':', label='x_max')\
"ax_out.scatter([t[first_peak_idx]], [x_max], color='tab:purple', zord
"\n",
"if np.isfinite(T_d):\n",
"    if len(max_idx) >= 2:\n",
"        t_p1 = t[max_idx[0]]\n",
"        t_p2 = t[max_idx[1]]\n",
"        y_arrow = max(x[max_idx[0]], x[max_idx[1]])\n",
"    else:\n",
"        t_p1 = t[first_peak_idx]\n",
"        t_p2 = t[first_peak_idx] + T_d\n",
"        y_arrow = x_max\n",
```

```
    "    ax_out.annotate('', xy=(t_p1, y_arrow), xytext=(t_p2, y_arrow), a
    "    ax_out.text((t_p1 + t_p2) / 2.0, y_arrow, 'T_d', color='tab:gray'
    "\n",
    "ax_out.set_title('sys1: PT2 identification from original impulse respo
    "ax_out.set_xlabel('$t/s$')\n",
    "ax_out.set_ylabel('$x(t)$')\n",
    "ax_out.grid()\n",
    "ax_out.legend(loc='upper left', bbox_to_anchor=(1.02, 1.0), borderaxes
    "\n",
    "annot_text = (\n",
    "    f'A_u = {A_u:.6f}\\n'\n",
    "    f'K_p = {K_p:.6f}\\n'\n",
    "    f'D = {D:.6f}\\n'\n",
    "    f'omega_0 = {omega_0:.6f} 1/s\\n'\n",
    "    f'T_d = {T_d:.6f} s\\n'\n",
    "    f't_max = {t_max_rel:.6f} s\\n'\n",
    "    f'x_max = {x_max:.6f}\\n'\n",
    "    f'x(inf) = {x_inf:.6f}'\n",
    ")\n",
    "ax_out.text(\n",
    "    0.02, 0.98, annot_text,\n",
    "    transform=ax_out.transAxes, va='top', ha='left',\n",
    "    bbox=dict(boxstyle='round', facecolor='white', alpha=0.85)\n",
    " )\n",
    "\n",
    "plt.tight_layout()\n",
    "plt.show()\n",
    "\n",
    "print('Estimated sys1 PT2 parameters from original impulse response:')
    "print(f'A_u = {A_u:.6f}')\n",
    "print(f'K_p = {K_p:.6f}')\n",
    "print(f'D = {D:.6f}')\n",
    "print(f'omega_0 = {omega_0:.6f} 1/s')\n",
    "print(f'T_d = {T_d:.6f} s')\n",
    "print(f't_max = {t_max_rel:.6f} s')\n",
    "print(f'x_max = {x_max:.6f}')\n",
    "print(f'x(inf) = {x_inf:.6f}')\n",
    "print('Transfer function (PT2 form):')\n",
    "print(f'G(s) = ({K_p:.6f}) / ((1/{omega_0:.6f}^2) s^2 + (2*{D:.6f}/{o
  ]
 },
 {
  "cell_type": "code",
  "execution_count": 128,
```

```
"id": "2d51d4e2",
"metadata": {},
"outputs": [
 {
  "data": {
   "image/png": "iVBORw0KGgoAAAANSUhEUgAABW4AAAJOCAYAAAAnP56mAAAAO
   "text/plain": [
    "<Figure size 1400x600 with 2 Axes>"
   ]
  },
  "metadata": {},
  "output_type": "display_data"
 },
 {
  "name": "stdout",
  "output_type": "stream",
  "text": [
   "Estimated sys2 IT1 parameters:\n",
   "K_I = 2.999842\n",
   "T_t = 0.010000 s\n",
   "T_1 = 0.482994 s\n",
   "Transfer function (IT1 with dead time):\n",
   "G(s) = (2.999842) / (s * (0.482994 s + 1)) * exp(-0.010000 s)\n"
  ]
 }
],
"source": [
 "# sys2 IT1 identification with dead time and annotated plot\n",
 "t = data2[:, 0]\n",
 "u = data2[:, 1]  # input y(t)\n",
 "x = data2[:, 2]  # output x(t)\n",
 "\n",
 "# Step detection on input (0 -> 2).\n",
 "trigger_candidates = np.where((u[:-1] < 1.0) & (u[1:] >= 1.0))[0]\n",
 "trigger_idx = trigger_candidates[0] + 1 if len(trigger_candidates) >
 "t_step = t[trigger_idx]\n",
 "\n",
 "# Input step height.\n",
 "u0 = np.mean(u[:max(1, trigger_idx)])\n",
 "u_inf = np.mean(u[trigger_idx:])\n",
 "du = u_inf - u0\n",
 "\n",
 "# Asymptotic tangent (lim t->inf) from last part of the measured respo
 "tail_start = int(0.75 * len(t))\n",
```

```
"if len(t) - tail_start < 5:\n",
"    tail_start = max(trigger_idx, len(t) - 5)\n",
"slope_inf, intercept_inf = np.polyfit(t[tail_start:], x[tail_start:],
"\n",
"# K_I from asymptotic slope: slope_inf = K_I * delta_u\n",
"K_I = slope_inf / du if abs(du) > 1e-12 else np.nan\n",
"\n",
"# Dead time estimate from sustained slope activation after the step.\
"dx_dt = np.gradient(x, t)\n",
"window = 5\n",
"kernel = np.ones(window) / window\n",
"dx_dt_smooth = np.convolve(dx_dt, kernel, mode='same')\n",
"post_idx = np.arange(trigger_idx, len(t))\n",
"slope_threshold = max(0.15 * slope_inf, 1e-6)\n",
"active_candidates = post_idx[np.where(dx_dt_smooth[post_idx] >= slope
"active_idx = active_candidates[0] if len(active_candidates) > 0 else
"T_t = t[active_idx] - t_step\n",
"\n",
"# Intersection of asymptotic tangent with time axis (x = 0).\n",
"t_intersect = -intercept_inf / slope_inf if abs(slope_inf) > 1e-12 el
"\n",
"# T_1 from tangent intersection relative to delayed step origin.\n",
"T_1 = t_intersect - (t_step + T_t) if np.isfinite(t_intersect) else n
"if np.isfinite(T_1):\n",
"    T_1 = max(T_1, 1e-6)\n",
"\n",
"# IT1 + dead-time model response to a step (for visual comparison).\n
"x0 = np.mean(x[:max(1, trigger_idx)])\n",
"tau = t - (t_step + T_t)\n",
"tau_pos = np.maximum(tau, 0.0)\n",
"if np.isfinite(T_1):\n",
"    x_model = x0 + K_I * du * (tau_pos - T_1 * (1.0 - np.exp(-tau_pos
"else:\n",
"    x_model = x0 + K_I * du * tau_pos\n",
"\n",
"# Asymptotic tangent line over plotting range.\n",
"tangent_inf = slope_inf * t + intercept_inf\n",
"\n",
"# Plot: left input, right output identification.\n",
"fig, (ax_in, ax_out) = plt.subplots(1, 2, figsize=(14, 6), gridspec_kw
"\n",
"ax_in.plot(t, u, color='tab:orange', label='y(t) input', linewidth=2)
"ax_in.axvline(t_step, color='black', linestyle='--', label='step origi
"ax_in.axhline(u_inf, color='tab:orange', linestyle=':', label='u(inf)
```

```
"ax_in.set_title('sys2 input signal')\n",
"ax_in.set_xlabel('$t/s$')\n",
"ax_in.set_ylabel('$y(t)$')\n",
"ax_in.grid()\n",
"ax_in.legend(loc='best')\n",
"\n",
"ax_out.plot(t, x, label='x(t) measured', linewidth=2)\n",
"ax_out.plot(t, x_model, ':', label='IT1 + dead-time model')\n",
"ax_out.plot(t, tangent_inf, '--', label='lim t->inf tangent')\n",
"\n",
"ax_out.axvline(t_step, color='black', linestyle='--', label='step orig
"ax_out.axvline(t_step + T_t, color='purple', linestyle='--', label='T
"if np.isfinite(t_intersect):\n",
"    ax_out.axvline(t_intersect, color='tab:green', linestyle='--', la
"    ax_out.scatter([t_intersect], [0.0], color='tab:green', zorder=5)
"\n",
"# Slope triangle on the asymptotic tangent: delta t = 1 s.\n",
"triangle_shift = 1.0\n",
"triangle_base = t_step + T_t + 0.5 + triangle_shift\n",
"t_tri0 = max(triangle_base, t[0])\n",
"t_tri1 = min(t_tri0 + 1.0, t[-1])\n",
"if t_tri1 - t_tri0 < 1e-9:\n",
"    t_tri0 = max(t[-1] - 1.0, t[0])\n",
"    t_tri1 = t[-1]\n",
"x_tri0 = slope_inf * t_tri0 + intercept_inf\n",
"x_tri1 = slope_inf * t_tri1 + intercept_inf\n",
"\n",
"ax_out.plot([t_tri0, t_tri1], [x_tri0, x_tri0], color='tab:blue', line
"ax_out.plot([t_tri1, t_tri1], [x_tri0, x_tri1], color='tab:blue', line
"ax_out.text((t_tri0 + t_tri1) / 2.0, x_tri0, '1', color='tab:blue', h
"ax_out.text(t_tri1, (x_tri0 + x_tri1) / 2.0, r'$K_I$', color='tab:blue
"\n",
"# Keep readable scale from measured output.\n",
"y_min = np.min(x)\n",
"y_max = np.max(x)\n",
"y_pad = 0.1 * (y_max - y_min) if y_max > y_min else 1.0\n",
"ax_out.set_xlim(t[0], t[-1])\n",
"ax_out.set_ylim(y_min - y_pad, y_max + y_pad)\n",
"\n",
"ax_out.set_title('sys2: IT1 identification with marked parameters')\n
"ax_out.set_xlabel('$t/s$')\n",
"ax_out.set_ylabel('$x(t)$')\n",
"ax_out.grid()\n",
"ax_out.legend(loc='upper left', bbox_to_anchor=(1.02, 1.0), borderaxes
```

```
    "\n",
    "# Annotate K_I and T_1 on the output plot.\n",
    "annot_text = f'K_I = {K_I:.6f}\\nT_1 = {T_1:.6f} s\\nT_t = {T_t:.6f}
    "ax_out.text(0.02, 0.98, annot_text, transform=ax_out.transAxes, va='to
    "\n",
    "plt.tight_layout()\n",
    "plt.show()\n",
    "\n",
    "print('Estimated sys2 IT1 parameters:')\n",
    "print(f'K_I = {K_I:.6f}')\n",
    "print(f'T_t = {T_t:.6f} s')\n",
    "print(f'T_1 = {T_1:.6f} s')\n",
    "print('Transfer function (IT1 with dead time):')\n",
    "print(f'G(s) = ({K_I:.6f}) / (s * ({T_1:.6f} s + 1)) * exp(-{T_t:.6f}
   ]
  },
  {
   "cell_type": "markdown",
   "id": "74b9d283",
   "metadata": {},
   "source": [
    "#### Calculating the system deviation from the optimal system paramet
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "fd91ea9e",
   "metadata": {},
   "outputs": [],
   "source": []
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "629e9b65",
   "metadata": {},
   "outputs": [],
   "source": []
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "3714fd67",
```

```
   "metadata": {},
   "outputs": [],
   "source": []
  },
  {
   "cell_type": "markdown",
   "id": "b9bbe9e1",
   "metadata": {},
   "source": [
    "#### Section to plot the Regeungstechnische and Simulink Model each sy
   ]
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "bf249eee",
   "metadata": {},
   "outputs": [],
   "source": []
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "4c41c029",
   "metadata": {},
   "outputs": [],
   "source": []
  },
  {
   "cell_type": "code",
   "execution_count": null,
   "id": "76dbc531",
   "metadata": {},
   "outputs": [],
   "source": []
  }
 ],
 "metadata": {
  "kernelspec": {
   "display_name": "base",
   "language": "python",
   "name": "python3"
  },
  "language_info": {
```

```
    "codemirror_mode": {
     "name": "ipython",
     "version": 3
    },
    "file_extension": ".py",
    "mimetype": "text/x-python",
    "name": "python",
    "nbconvert_exporter": "python",
    "pygments_lexer": "ipython3",
    "version": "3.13.5"
   }
  },
  "nbformat": 4,
  "nbformat_minor": 5
}
```

B.2 Code used for the analysis of the Experiment

C Bibliography